Application Security

Do / Say / See

Repository scan, attack surface, finding evidence and an existing proposed fix.

4 pillars | 14 core pages | limit 20

Use the existing demo-site-pr-examples scan and SQL finding. The completed fix is PR #7. This guide follows the final Version 3 video with its 21.14-second introduction. All timestamps include that introduction.

Read each grouped step top to bottom. Do contains the clicks and scrolls; read only Say aloud. See is a frame from the delivered video. Times refer to that video. The narration is preserved in full, including the recorded opening and closing.

Creation, Send and approval actions describe the recorded flow. Rehearse with the linked existing results; use a separate training run when repeating those actions. Live pages may have changed. Resource links require the corresponding account access.

OPEN THE DEMO

Example repository

Deep Scan finding: CWE-89

Recorded security-fix task

Security-fix PR #7

PILLARS AND PAGE LIMITS

Introduction - 1 page

Deep Scan - 2 steps / pages

Explore Attack Surface - 3 steps / pages

Investigate a Finding - 4 steps / pages

Fix with AI - 4 steps / pages

The core count includes the opening page when present. This cover, operator prompts and resource directory are outside the core. Every pillar uses at most four grouped steps.

INTRODUCTION / OPENING / 00:00-00:19

Introduce the AppSec flow

DO

Show the Application Security flow slide. Introduce the four pillars before entering the existing scan.

SAY

Let's look at security from the AppSec team's perspective. We'll start with the deep scan of the repository, explore the attack surface, and investigate a finding by following the evidence back to the code. Then we'll look at how Fix with AI turns that finding into a proposed patch. With test results, the team can inspect before deciding what to merge.

SEE / VIDEO FRAME AT 00:09



DEEP SCAN / STEP 1 OF 2 / 00:21-00:38

Open the repository scan setup

DO

Open CodeRabbit Security and select coderabbit-demo/demo-site-pr-examples.

Click Run AI Deep Scan and inspect the selected repository and setup options.

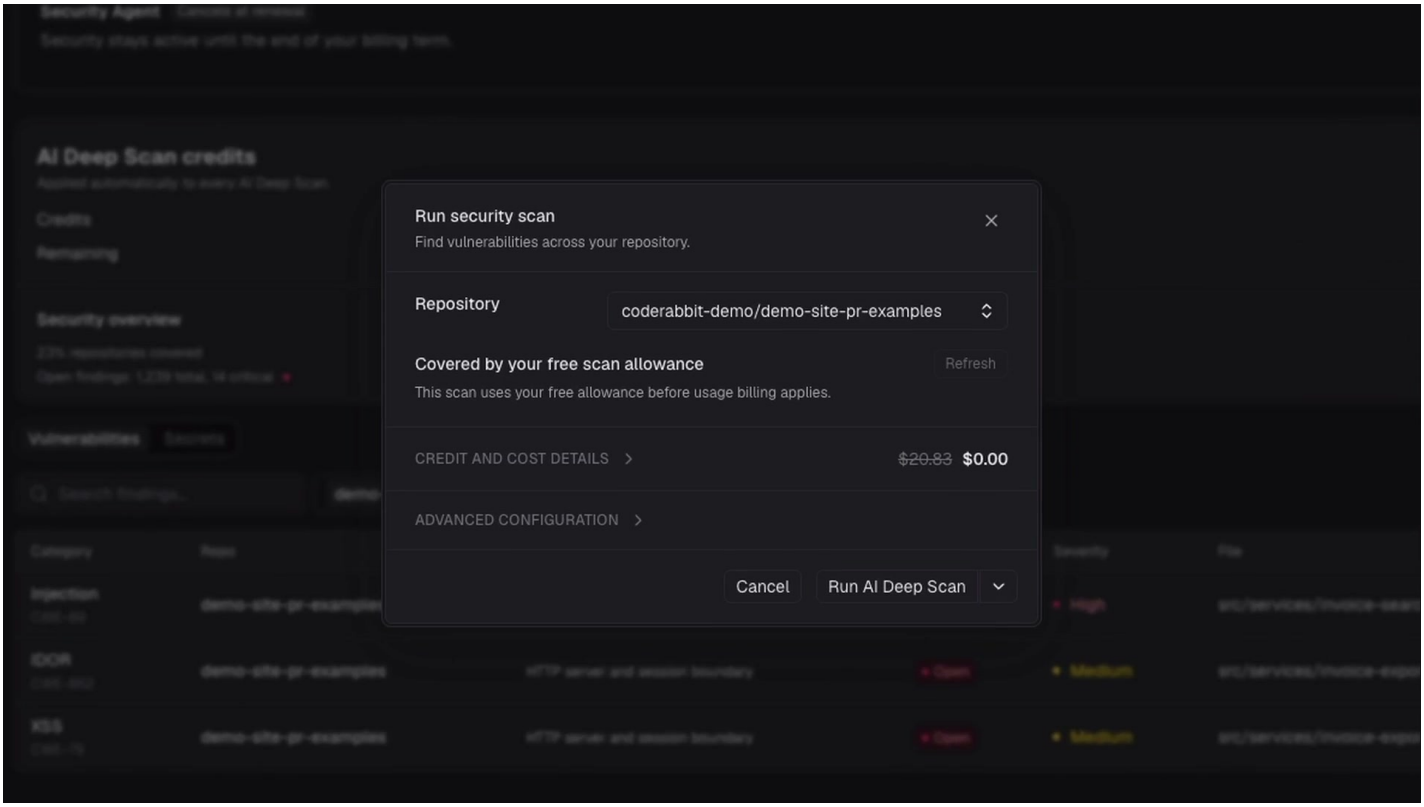
Deep Scan finding: CWE-89

Example repository

SAY

We'll start with an application security review of this repository. The question is where customer data could be exposed and what evidence we have to investigate that risk. An AI deep scan examines committed code across the repository. Here we can choose the repository and review the scan setup.

SEE / VIDEO FRAME AT 00:32



DEEP SCAN / STEP 2 OF 2 / 00:39-00:43

Use the existing analysis

DO

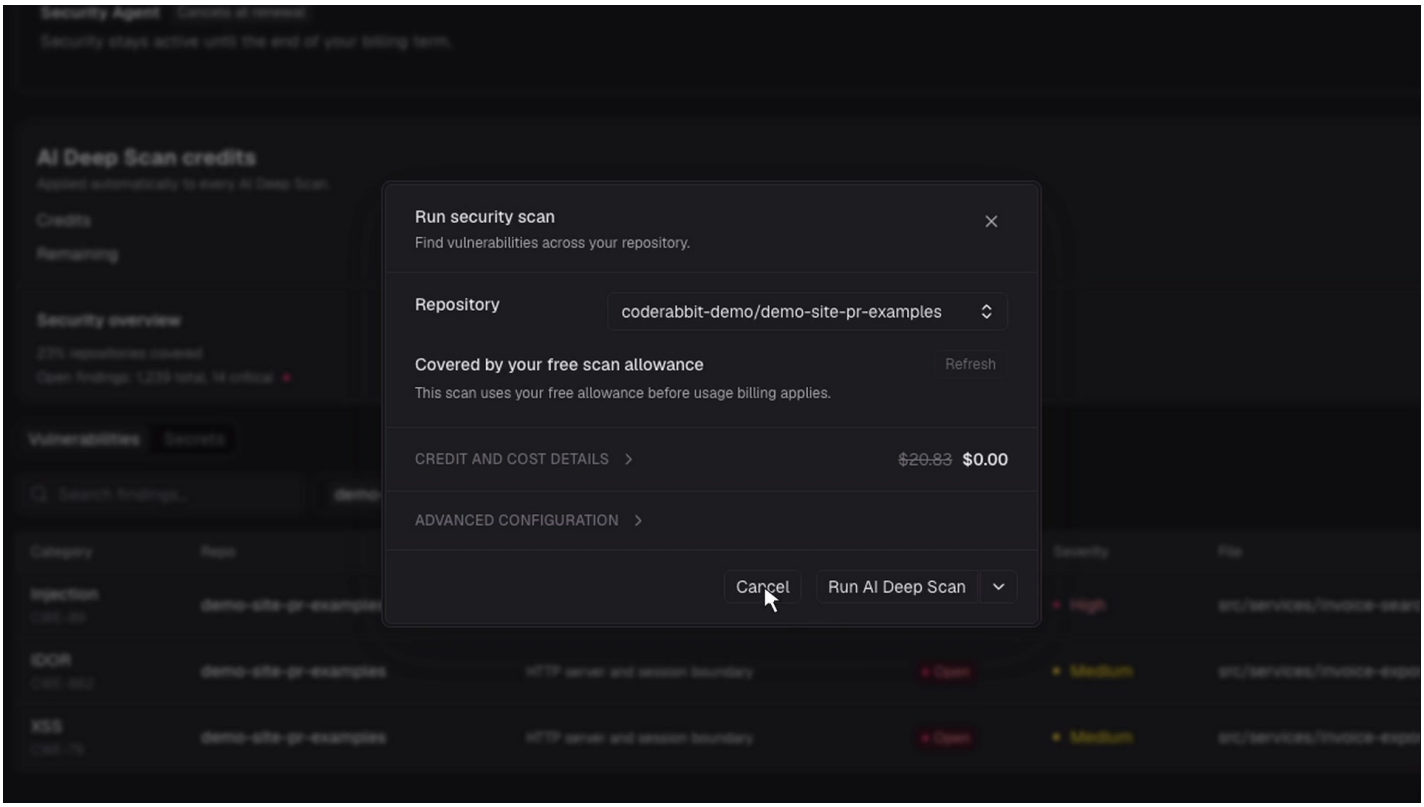
Click Cancel or close the setup dialog.

Keep the existing scan results visible; this demonstration does not start a new scan.

SAY

This demo already has scan results so I'll close the setup and use the existing analysis.

SEE / VIDEO FRAME AT 00:42



EXPLORE ATTACK SURFACE / STEP 1 OF 3 / 00:44-00:58

Open the application map

DO

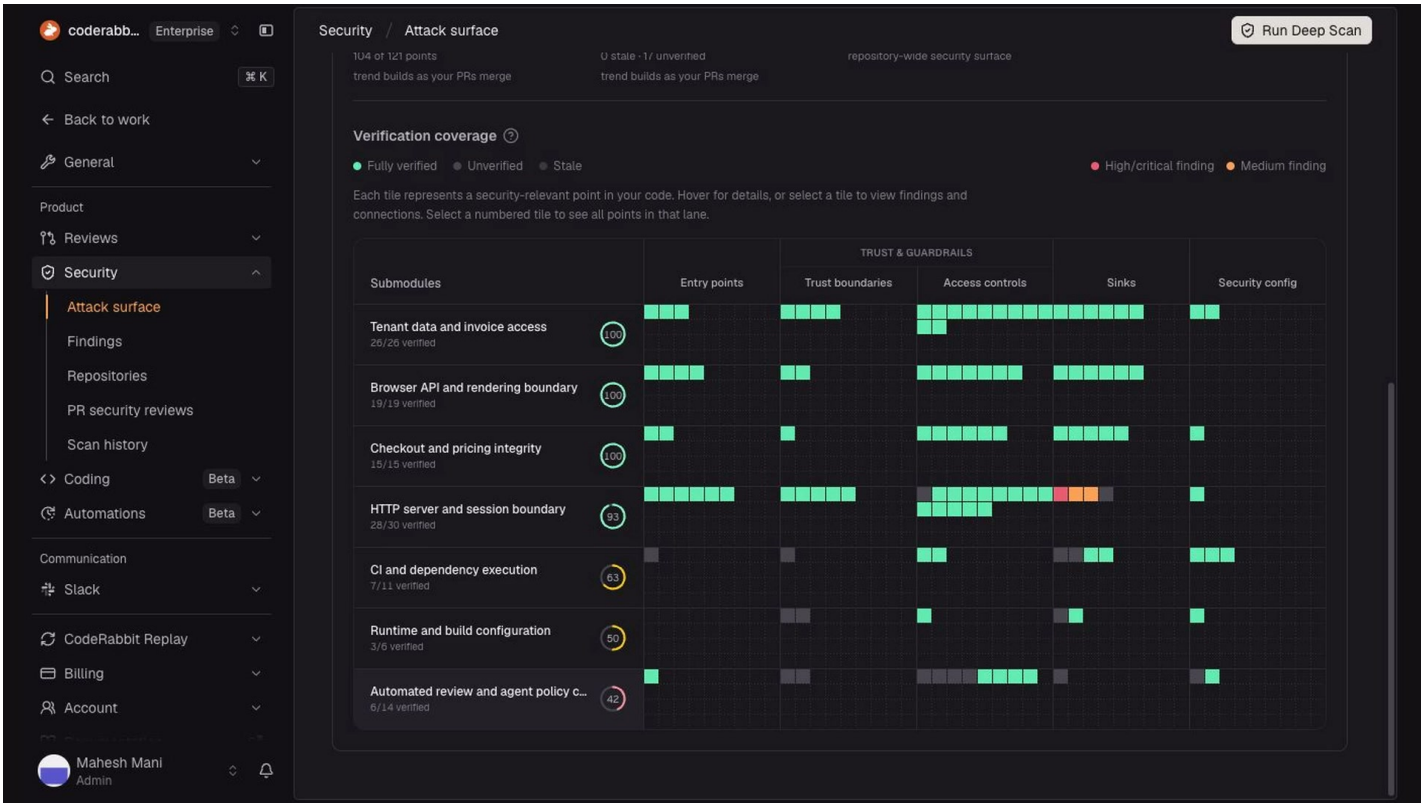
Click Attack Surface in the Security navigation.

Scroll or fit the map to show entry points, controls and sensitive operations.

SAY

Let's open attack surface. This gives us a view of the application's security relevant components and how they connect. We can use it to explore where requests enter, which controls apply and where sensitive operations happen.

SEE / VIDEO FRAME AT 00:54



EXPLORE ATTACK SURFACE / STEP 2 OF 3 / 00:59-01:23

Trace invoice search and tenant isolation

DO

Select the invoice-search component in the map.

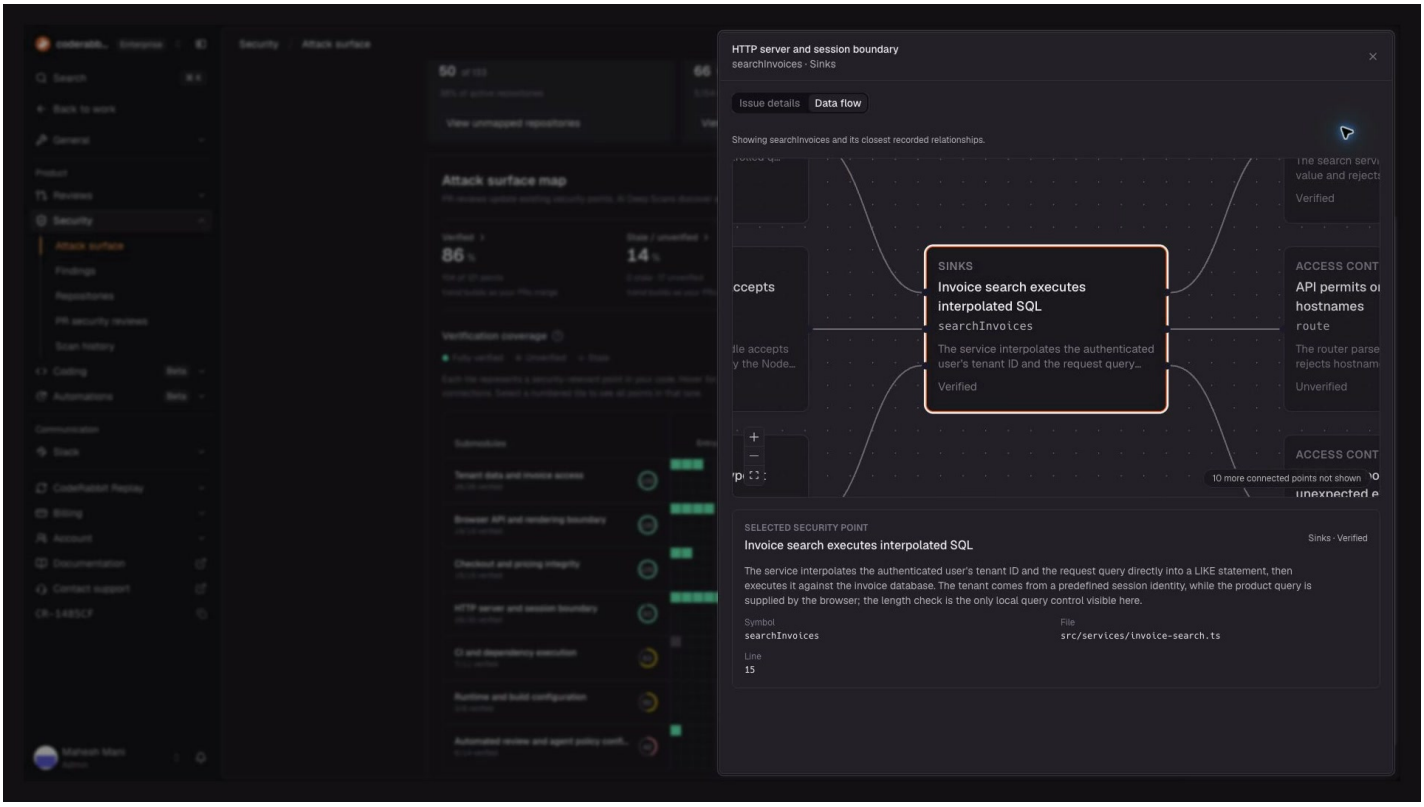
Open its supporting evidence and scroll to authentication, search input and the database query.

Keep the customer boundary in view.

SAY

For this example, we'll focus on invoice search. A customer signs in, sends a search request and receives matching invoices. The important question is whether those results stay within that customer's tenant. Authentication is one part of that path. The database query also needs to enforce the boundary between the customers. I'll use the map and its supporting evidence to examine how the request reaches that query.

SEE / VIDEO FRAME AT 01:15



EXPLORE ATTACK SURFACE / STEP 3 OF 3 / 01:24-01:33

Check the analyzed scope

DO

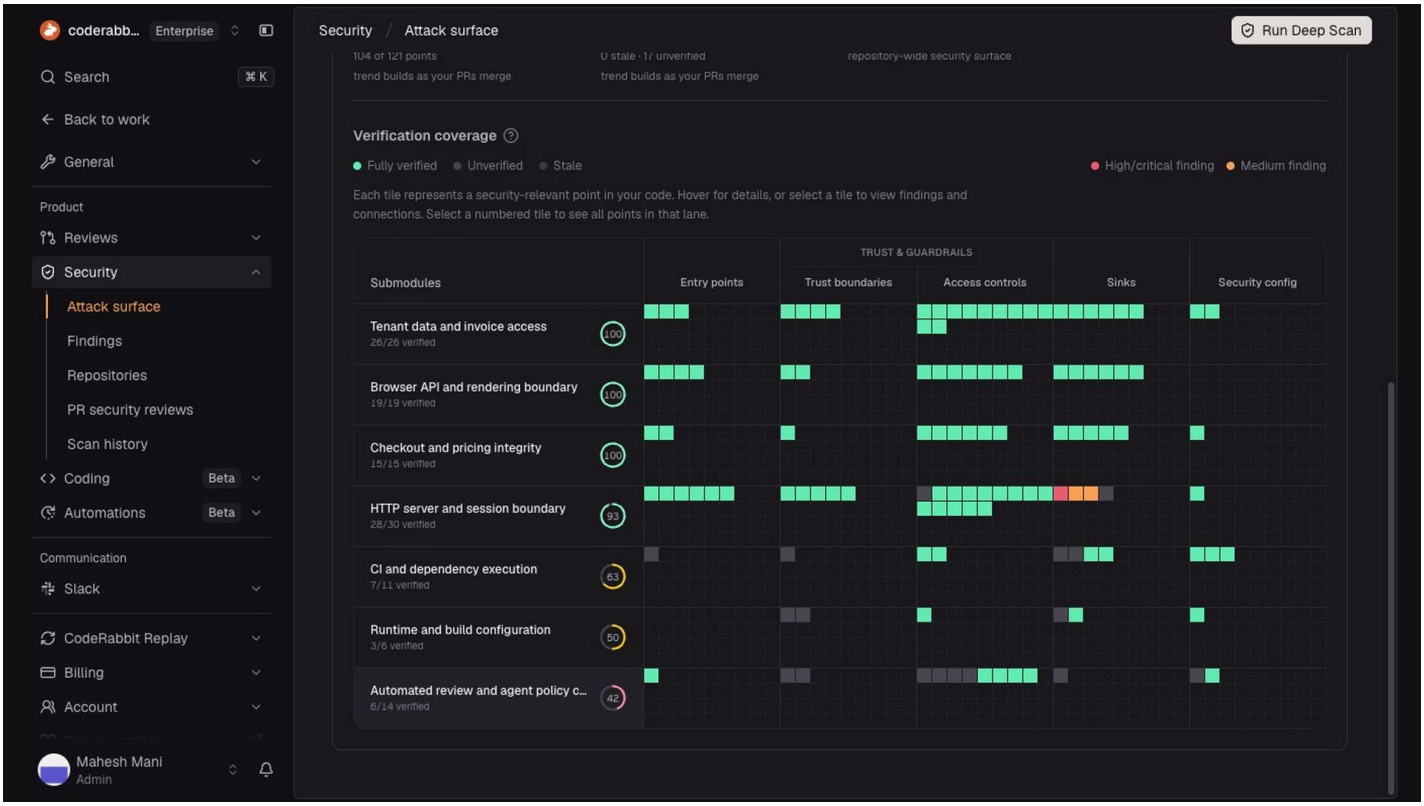
Close the component detail panel and read the map's coverage/analysis context.

Keep the existing scan's code scope distinct from later repository changes.

SAY

The scan results describe the code that was analyzed. As we investigate, we need to keep that scope in mind, especially if the repository has changed since the scan.

SEE / VIDEO FRAME AT 01:30



INVESTIGATE A FINDING / STEP 1 OF 4 / 01:34-01:51

Open the SQL finding and priority evidence

DO

Click Findings in the Security sidebar. Select the same repository and the SQL injection finding.

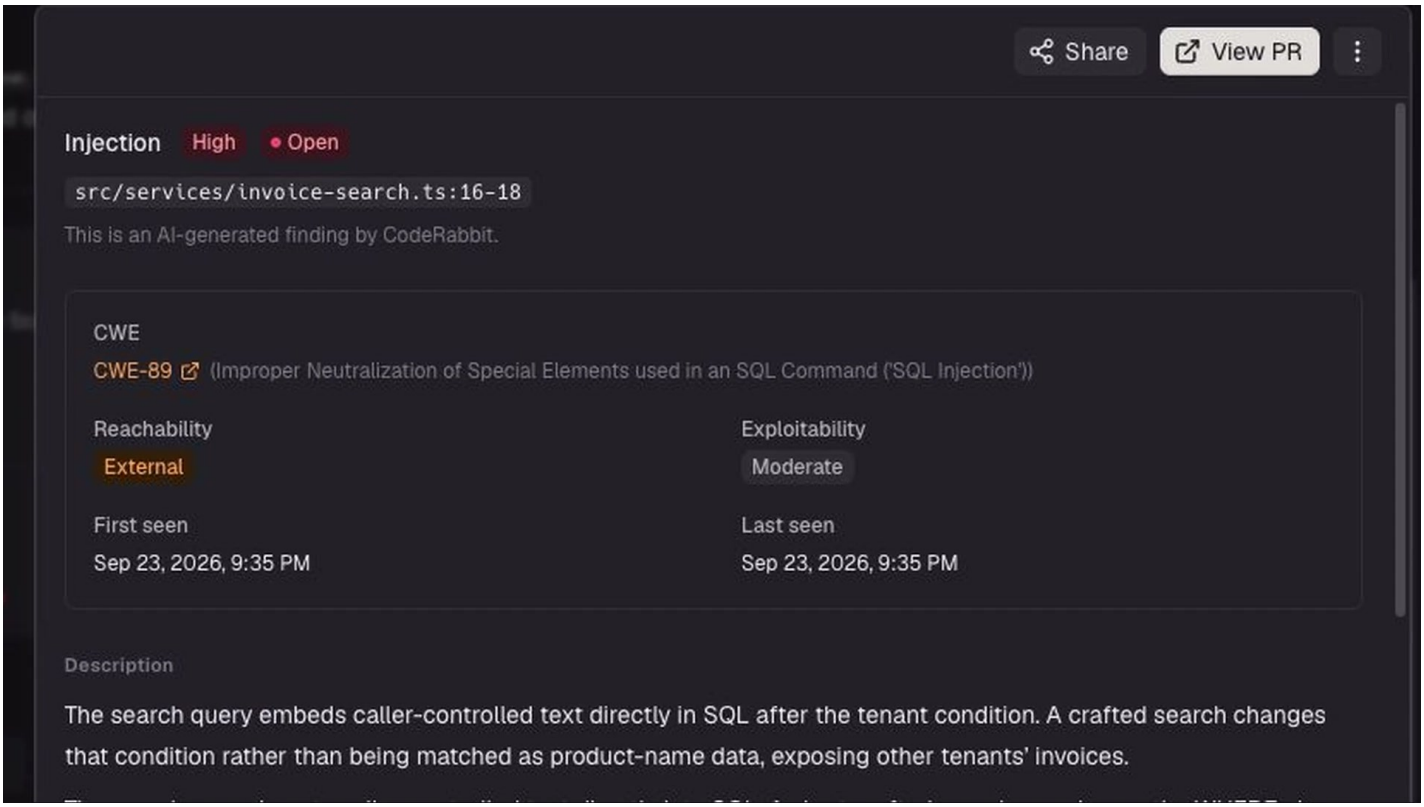
Read High severity, Open, reachability and exploitability.

Deep Scan finding: CWE-89

SAY

Now let's open findings for this repository and select the SQL injection finding. It's marked high severity and open. The affected code is an invoice search. The finding also shows reachability and exploitability, which helps us prioritize the investigation.

SEE / VIDEO FRAME AT 01:46



INVESTIGATE A FINDING / STEP 2 OF 4 / 01:52-02:19

Trace input into the SQL command

DO

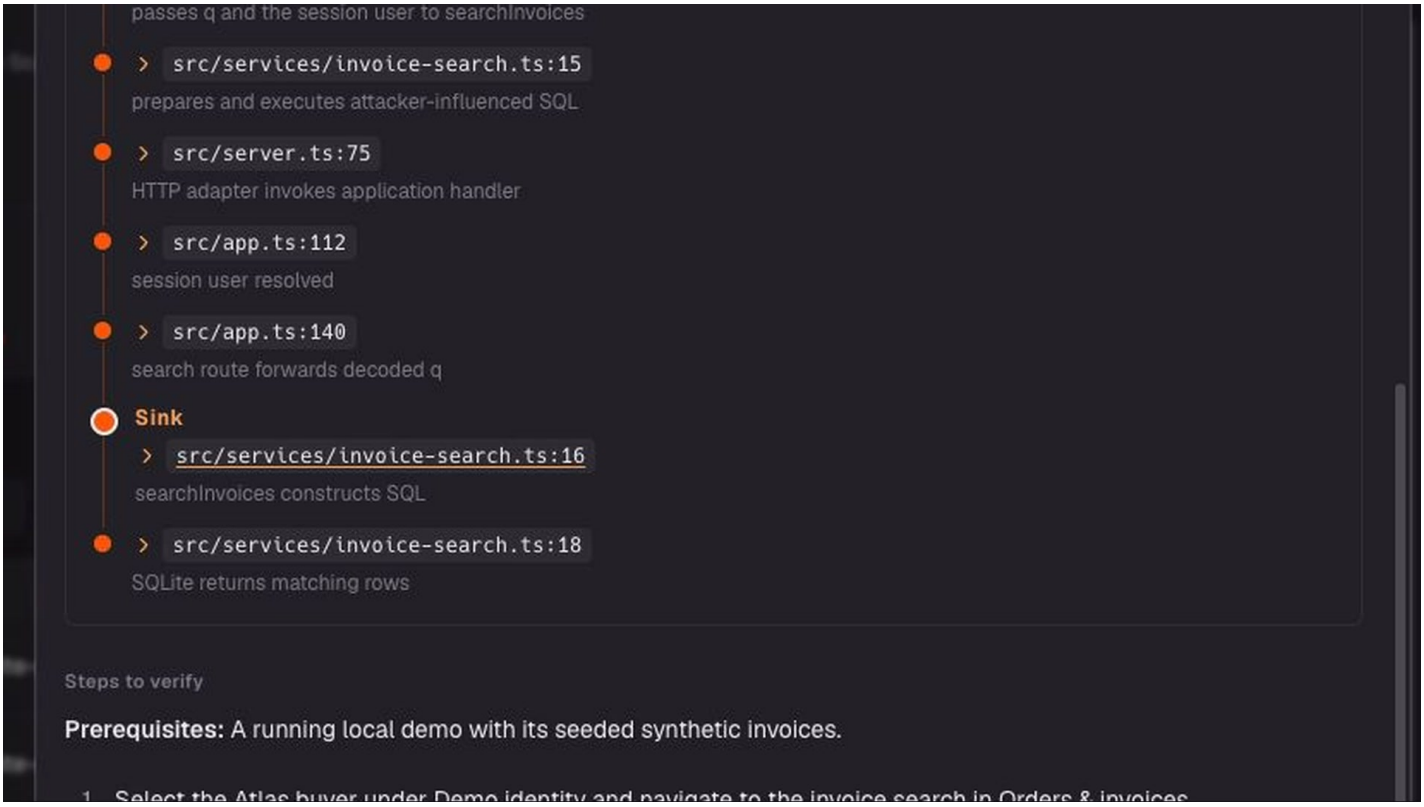
Scroll the detail panel to the evidence/call path.

Expand the cited request route and search function. Show search text and session user reaching the interpolated query.

SAY

Here's the evidence. Search text comes from authenticated invoice search request. The application passes that text and the session user into the search function. The function then inserts both values directly into a SQL command. A crafted search can change the command and bypass the tenant condition. In this demo, that could expose an invoice belonging to another customer, which even though the request came from a signed-in user.

SEE / VIDEO FRAME AT 02:10



INVESTIGATE A FINDING / STEP 3 OF 4 / 02:20-02:32

Inspect cited code and reproduction

DO

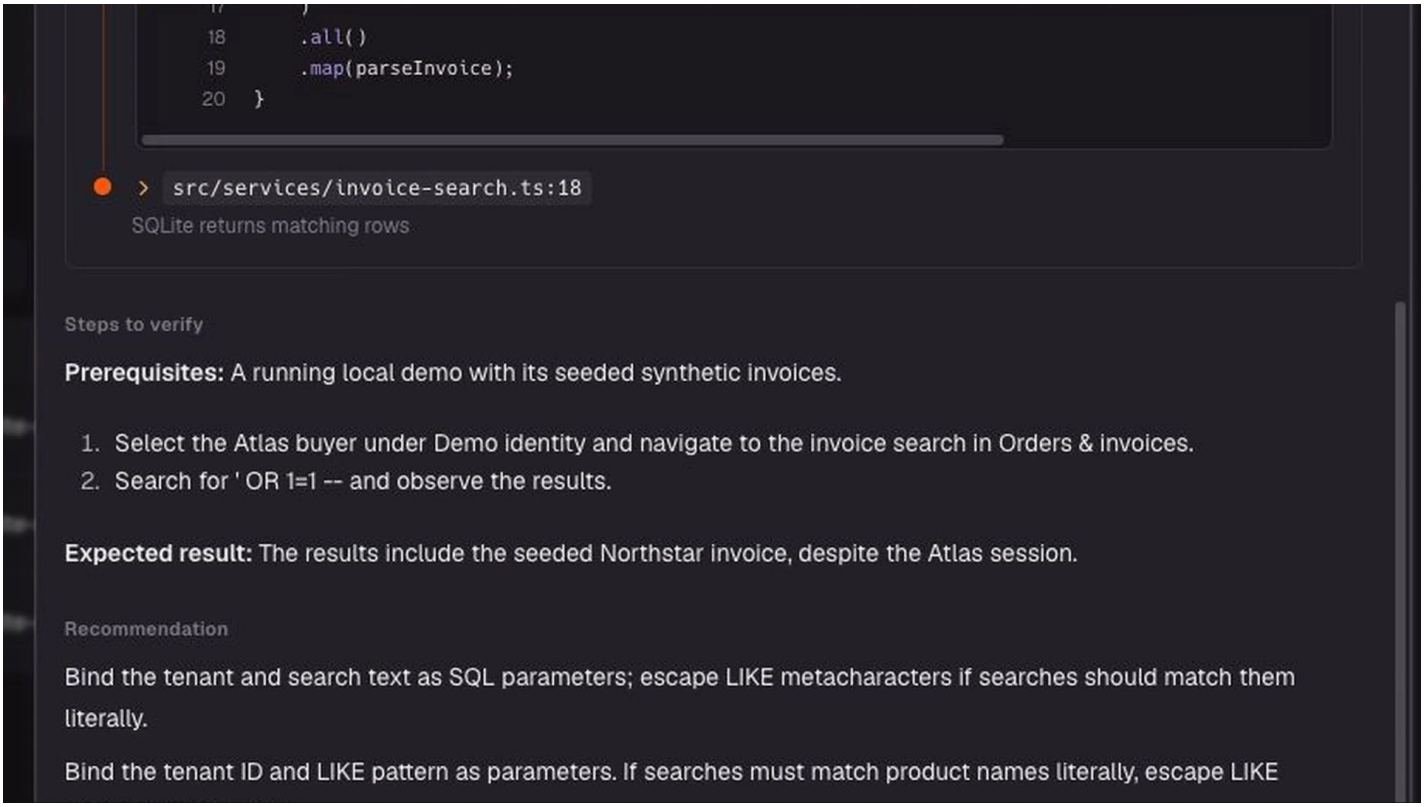
Open the cited lines and inspect invoice-search.ts.

Scroll the finding to the seeded-invoice reproduction and parameter-binding recommendation.

SAY

I can open the cited lines and inspect the query. The finding includes steps to reproduce the issue using the demo's seeded invoices, along with the recommendation to bind the tenant ID and search text as parameters.

SEE / VIDEO FRAME AT 02:28



INVESTIGATE A FINDING / STEP 4 OF 4 / 02:33-02:39

Read the proposed correction

DO

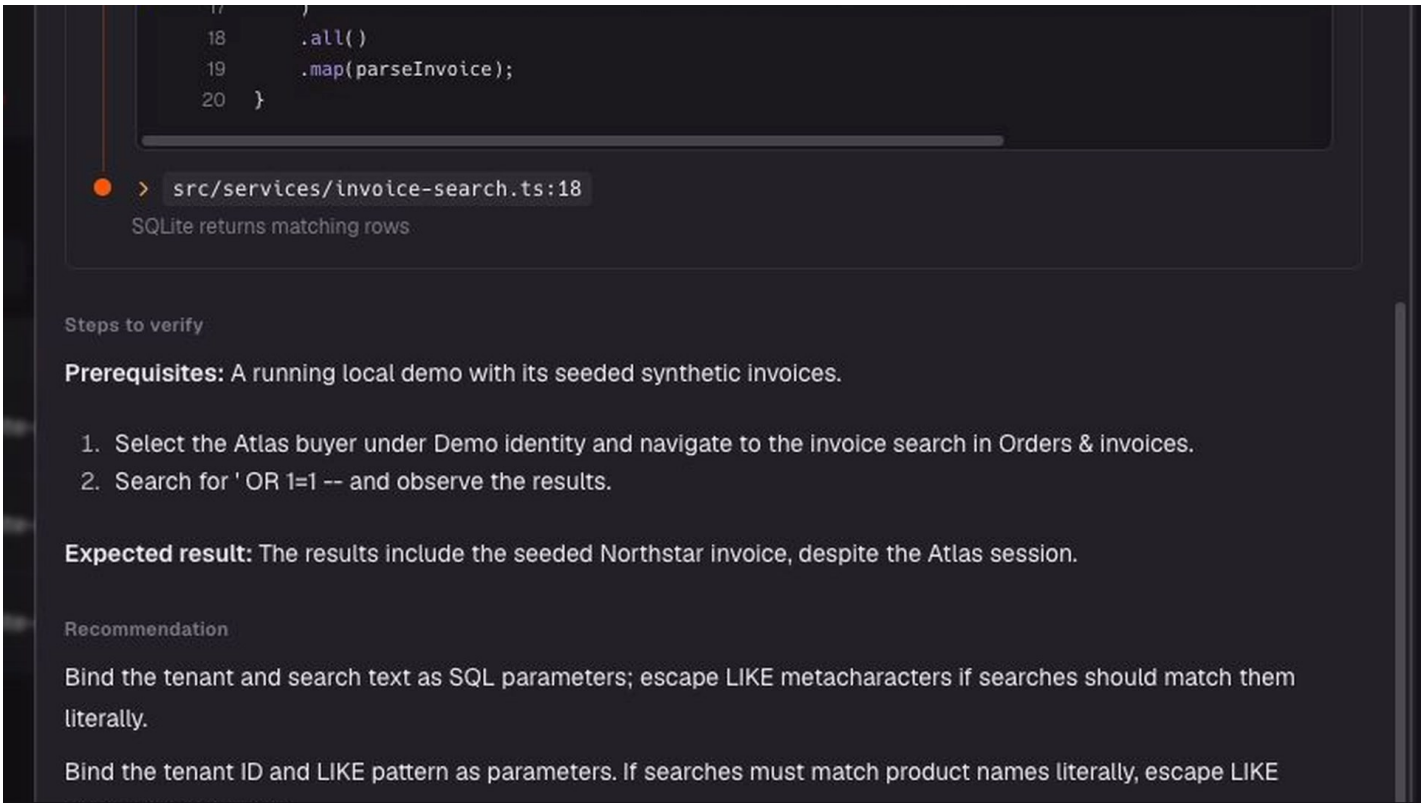
Keep Recommendation expanded.

Show the SQL placeholders and separately supplied tenant/search values.

SAY

That means passing the values separately from the SQL command, so the database treats them as data.

SEE / VIDEO FRAME AT 02:37



FIX WITH AI / STEP 1 OF 4 / 02:40-02:55

Open the existing result

DO

Scroll to the finding's action bar. In this completed run the action is View PR.

Click View PR to open the proposed fix; use the existing task link for the original context.

Deep Scan finding: CWE-89

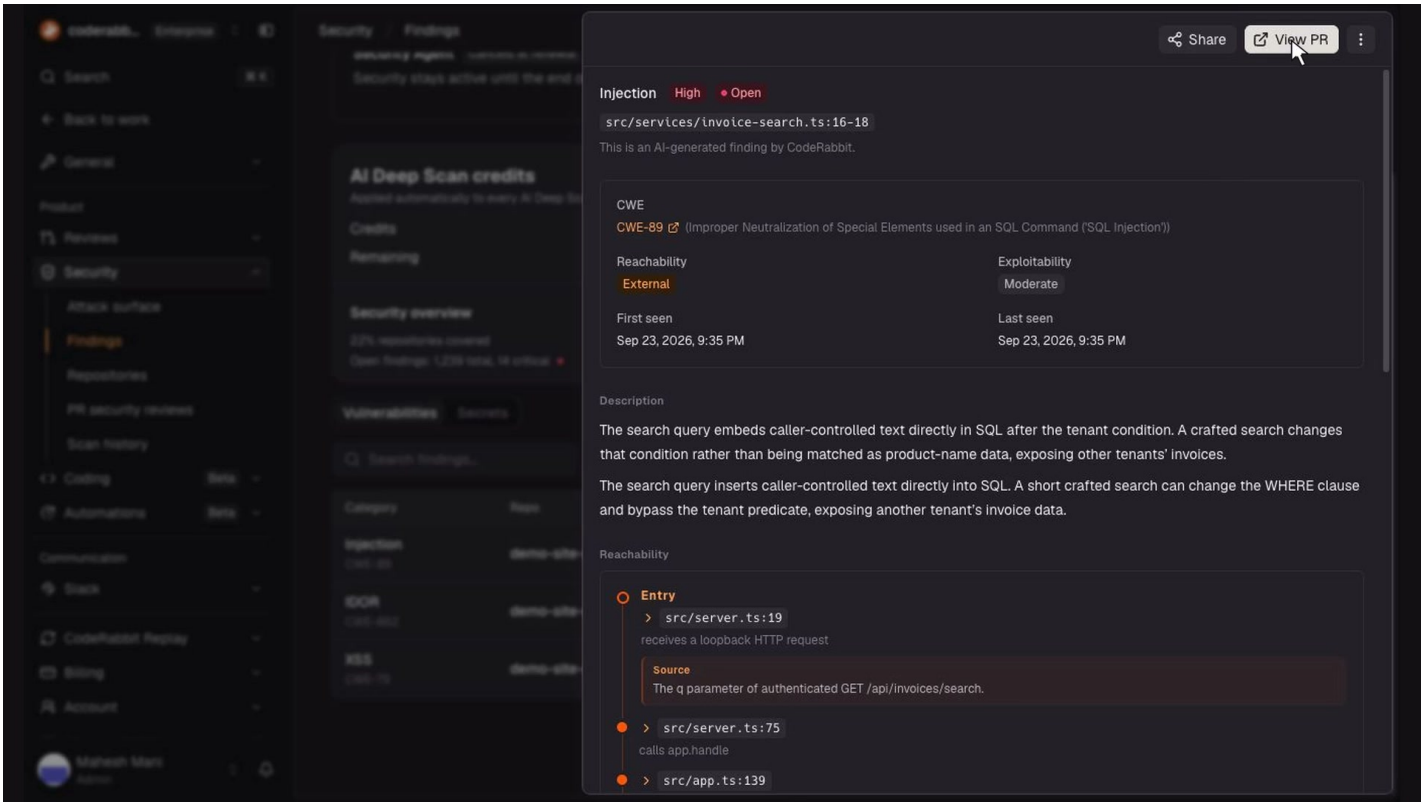
Recorded security-fix task

Security-fix PR #7

SAY

Next is Fix with AI. This can turn the finding and its supporting context into a coding task for a proposed fix. We've already completed the task for this finding, so the action now says view PR. Let's open the proposed change and look at the result.

SEE / VIDEO FRAME AT 02:50



FIX WITH AI / STEP 2 OF 4 / 02:56-03:05

Inspect query and test changes

DO

On PR #7, click Files changed. Select src/services/invoice-search.ts.

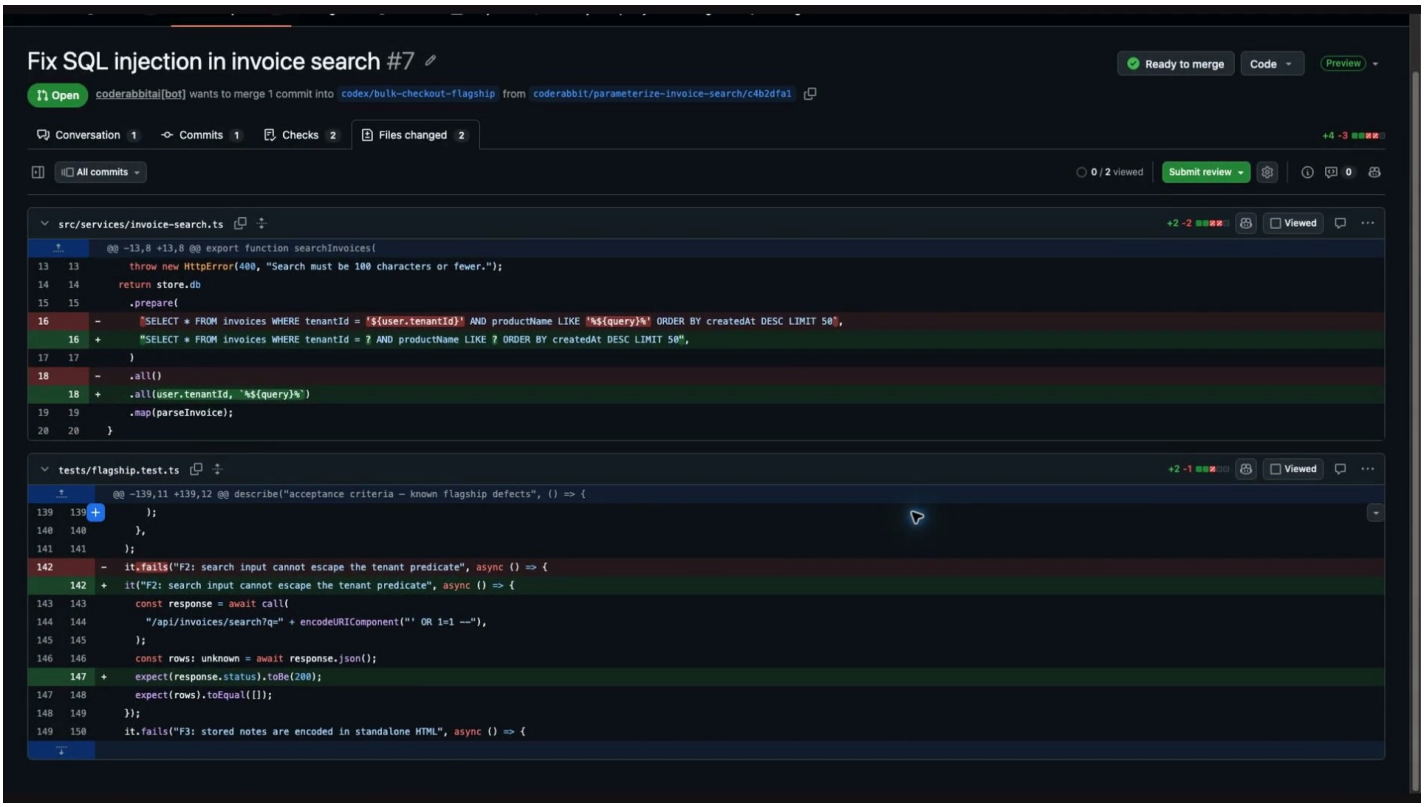
Show bound parameters, then open the customer-isolation test and its normal assertion replacing it.fails.

PR #7: files changed

SAY

The patch replaces the interpolated values with query parameters. The customer isolation test also changes from an expected failure to a normal regression test.

SEE / VIDEO FRAME AT 03:02



FIX WITH AI / STEP 3 OF 4 / 03:06-03:19

Read the reported test result

DO

Open the completed coding task and scroll to its final summary.

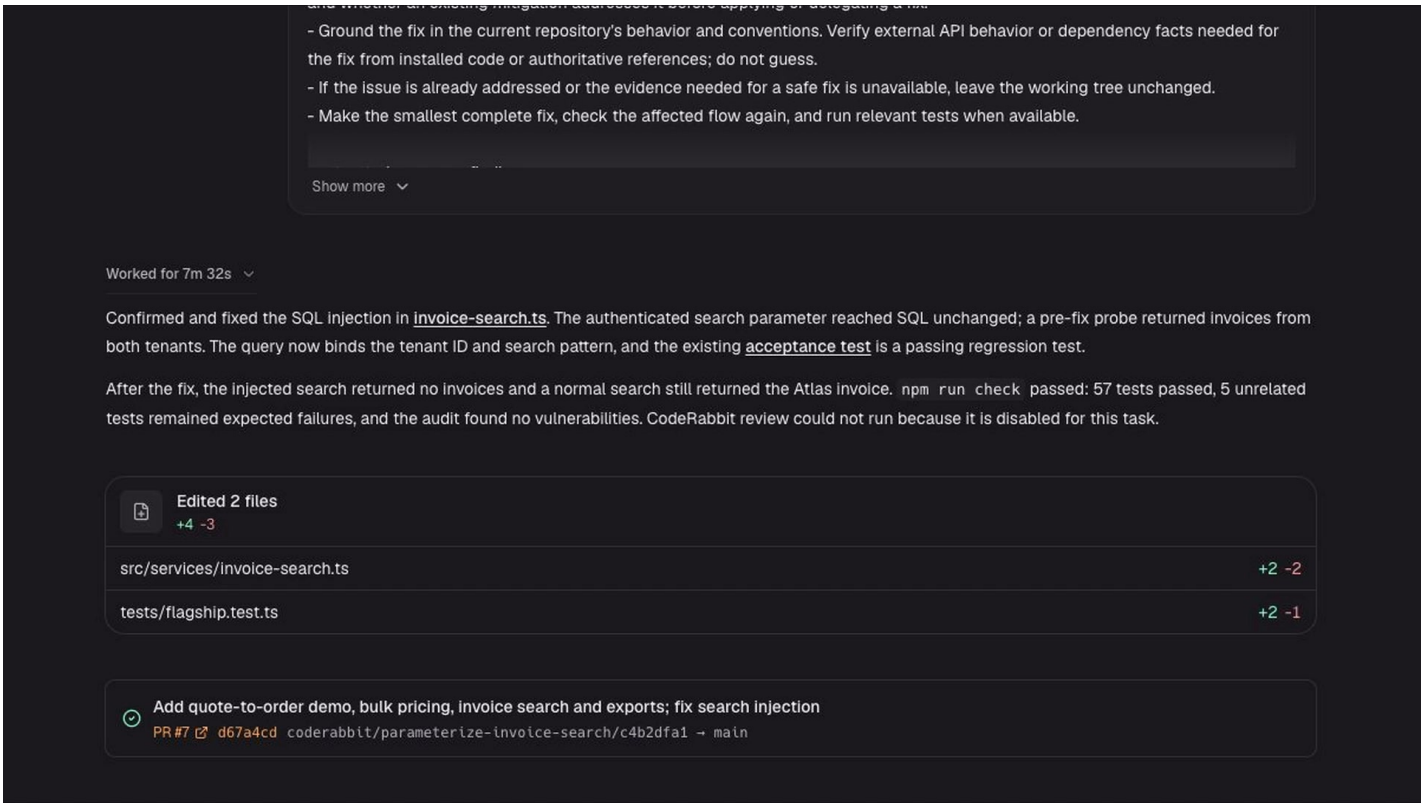
Read the injection/normal-search result and 57 passing tests with five expected failures.

Recorded security-fix task

SAY

The coding task reports that the injection attempt no longer returns another customer's invoices, while a normal search still works. It reports 57 passing tests with 5 unrelated expected failures.

SEE / VIDEO FRAME AT 03:15



FIX WITH AI / STEP 4 OF 4 / 03:21-03:43

Keep the fix reviewable and finish

DO

Return to PR #7 and inspect the patch/checks. Leave it open and unmerged.

Finish the narration and show the closing application-security flow slide.

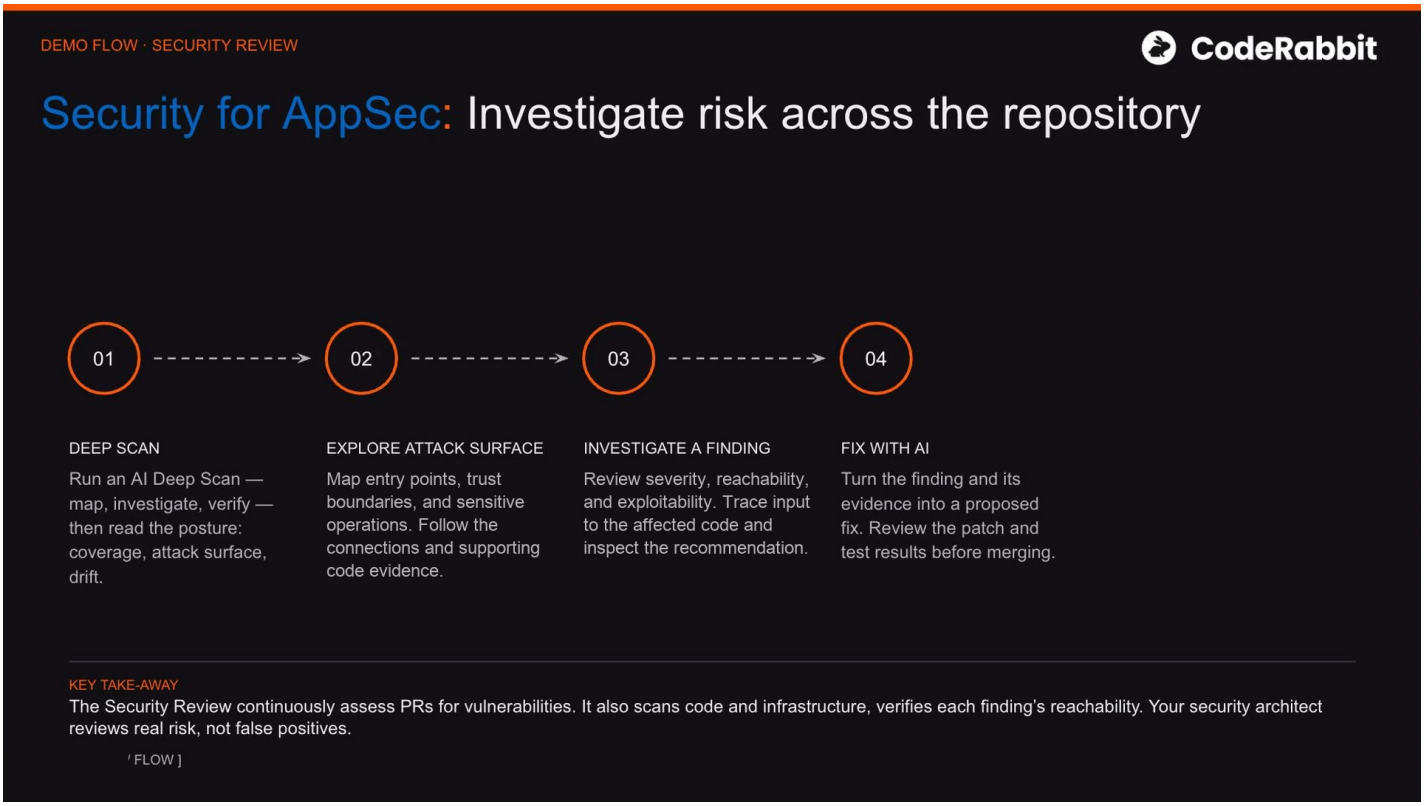
Security-fix PR #7

PR #7: files changed

SAY

Those results give us something concrete to review. We still need to inspect the patch and the checks before accepting the change. For this demo, we'll leave the proposed fix unmerged. We've gone from a repository scan to the application's attack surface, followed a finding back to its code, and reviewed a proposed fix. That's the AppSec workflow we are showing here.

SEE / VIDEO FRAME AT 03:37



Resource directory

Links open the recorded examples. Historical comments and pinned source revisions preserve the relevant context when the live PR has moved on.

Example repository

https://github.com/coderabbit-demo/demo-site-pr-examples

Deep Scan finding: CWE-89

https://app.coderabbit.ai/security/code?owner=coderabbit-demo&repo=demo-site-pr-examples&findingId=5bbe54d1-f97b-48ba-9a95-9cc1300c4cdf

Recorded security-fix task

https://app.coderabbit.ai/code/tasks/c4b2dfa1-bb4c-4d5d-b959-f1f6ea606cfb

Security-fix PR #7

https://github.com/coderabbit-demo/demo-site-pr-examples/pull/7

PR #7: files changed

https://github.com/coderabbit-demo/demo-site-pr-examples/pull/7/files